Instituto Superior de Formación Técnica Nº 151
Carrera: Analista en Sistemas
3 Año. Algoritmos y Estructuras de Datos III.
Alumna: LUZURIAGA, Silvana L.

COMPLEJIDAD ALGORITMICA

Marco Teórico:

1. Describir el Concepto de Complejidad en el tiempo

La complejidad en el tiempo es un concepto teórico que indica cuanto esfuerzo o costo en tiempo requiere un algoritmo para ejecutarse a medida que se incrementa el tamaño de los datos de entrada(*n*). No se mide en unidades de tiempo real (como segundos o nanosegundos), ya que el tiempo de ejecución real depende siempre del sistema (hardware, supercomputadora vs. PC, lenguaje de programación como C++ vs. Java o si es compilado o interpretado). En su lugar, se mide mediante unidades lógicas que expresan la relación o tasa de crecimiento del numero de operaciones que realiza la CPU en función del tamaño de la entrada.

2. Explicar el concepto del Ave, Cohete e Internet.

Esta es una alegoría clásica para ilustrar como difieren las tasas de crecimiento de los algoritmos ante el aumento de datos (*n*):

El Ave: representa un tiempo de ejecución constante, equivalente a *O* (1). No importa si el tamaño de los archivos aumenta drásticamente; el ave siempre tardara el mismo tiempo en llevar la información a destino (el tiempo no varía en función de *n*).

Internet: representa un tiempo de ejecución lineal, equivalente a *O* (1). El tiempo de llegada de los datos varia de forma directamente proporcional al tamaño del archivo (a mayor cantidad de megabytes, mas tiempo demandara la transferencia).

El Cohete: se utiliza para graficar algoritmos muy rápidos para volúmenes pequeños de datos, pero cuyo tiempo varia o escala drásticamente de manera perjudicial cuando el volumen de elementos*(n*) crece exponencial o polinomialmente (zonas de alta complejidad).

Conclusión de la alegoría: el algoritmo que crece menos en el tiempo ante entradas masivas es el mas eficiente (rendimiento = escala).

3. Que se entiende por notación asintótica, para que sirve

Las notaciones asintóticas son herramientas matemáticas que sirven para representar y evaluar la complejidad de ejecución (tanto en tiempo como en espacio) de los algoritmos sin depender de las constantes de una maquina especifica ni requerir la implementación previa del código para compararlos.

Para que sirven?

* Permite comparar algoritmos de forma puramente metodológica y matemática sin necesidad de implementarlos físicamente ni medir tiempos reales en segundos.
* Se enfoca en el comportamiento con grandes volúmenes de datos (n→ ∞), donde los factores de hardware ya no relevantes para determinar cuál algoritmo escala mejor.

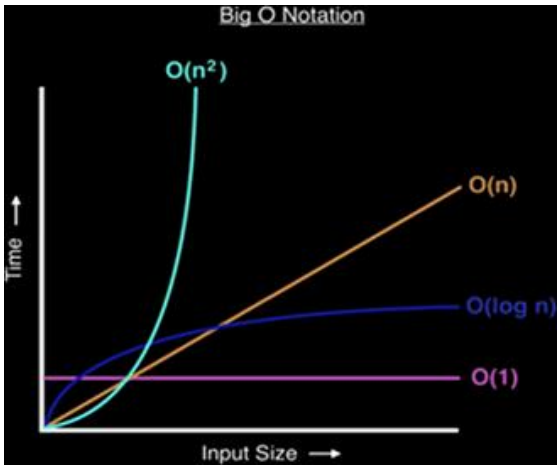
4. Describir, Big O, Big Omega y Big Theta
Son las tres herramientas matemáticas principales para acotar el comportamiento de una función:

Notación Big O (*O*): Define un **límite superior** asintótico de un algoritmo. Limita la función solo desde arriba: f(*n*) -< c * g(*n*)) para todo *n*-> n_0. Se utiliza para calcular el **peor de los casos**, garantizando que el algoritmo nunca tardara más que ese límite.

Notación Omega (Ω): Define el **limite inferior** asintótico. Limita la función desde abajo f(*n*) -> c * g(*n*)) representando el **mejor de los casos** (el número mínimo de operaciones). Es la menos utilizada porque asegurar un mínimo de velocidad no ayuda si en el peor escenario tarda años.

Notación Big Theta (Θ): Define el **comportamiento asintótico exacto**, **limitando** la función tanto desde arriba como desde abajo c_1 * g(n) -< = f(n) -< c_2 * g(n)). Se da cuando el mejor y el peor caso de un algoritmo son asintóticamente iguales.

5. Explicar la siguiente Gráfica de Big O



Esta grafica ilustra como varían las distintas curvas de complejidad a medida que crece el tamaño de la entrada (Input Size) en el eje horizontal frente al tiempo/operaciones en el eje vertical:

1- *O*(1) (línea rosa plana, Constante): Una línea completamente horizontal, tiempo constante. El tiempo no cambia sin importar el tamaña de *n* o sea la entrada.
2- *O*(log *n*) (Curva azul baja, Logarítmico): tiempo logarítmico, crecimiento sumamente lento y controlado (como en la búsqueda binaria). Altamente eficiente para grandes volúmenes o masivas.

3- *O*(*n*) (línea naranja diagonal, Lineal): tiempo lineal. El numero de operaciones crece de forma proporcional a n (búsquedas simples o bucles únicos). Crece en proporción directa 1 a 1 con el tamaño de los datos.

4- *O*(*n* log *n*) (curva verde verticalizada, Linealitmico): tiempo cuadratico. Para entradas pequeñas compite con las otras, pero al pasar un umbral critico (n_0) su crecimiento se dispara exponencialmente hacia arriba, entrando en lo que la teoría llama la "*Zona Mala*" de rendimiento.

6. Explicar la diferencia entre Complejidad en el Tiempo y Complejidad Espacial.

*** Complejidad en el Tiempo:** mide la cantidad de **tiempo/operaciones de CPU** que requiere un programa para ejecutarse hasta completarse en función del tamaño de entrada. Su prioridad es la velocidad y la escala.

***Complejidad Espacial**: mide la cantidad de **memoria física** (almacenamiento) que el algoritmo necesita en el sistema para ejecutarse. Incluye tanto el espacio ocupado por los datos de entrada como el espacio extra o auxiliar (por ejemplo, la memoria utilizada por variables adicionales o pila de llamadas en una recursión).

Marco Practico:

1. **Determinar y crear la Función Big O para los siguientes Códigos:**

Fragmento 1:
Python

```
for i=0 to N;
   if i%2 == 0:
      print i
```

Análisis: el bucle itera de forma lineal exactamente *N* veces. Adentro hay operaciones constantes (modulo y asignación/salida) que toman *O* (1).

Función*: N* x *O* (1) = *O* (0).

Complejidad: *O* (*N*) (Tiempo Lineal).

Fragmento 2:
Python

```
for i = 0 to N:
   if i % 2 == 0:
      print i
for i = 0 to N:
    if i % 2 != 0:
      print i
```

Análisis: tenemos dos bucles independientes secuenciales (uno atrás del otro). Cada uno se ejecuta *N* veces.

Función: *O (N) = O (N) = O (2N).* Aplicando la regla de ignorar constantes, descartamos el 2.

Complejidad: *O (N)* (Tiempo Lineal):

Fragmento 3:
Python

```
for i = 0 to N:
  for j = i to N:
    print i + “,“ + j
```

Análisis: son bucles anidados. El bucle externo corre N veces. El bucle interno arranca en i, por lo que hace N operaciones la primera vez, N -1 la segunda, hasta 1. Matemáticamente, esto equivale a (Nx (N+1)) /2 = $N^2/2$ + N/2. Eliminamos el termino no dominante (N/2) y la constante multiplicada (1/2).

Función: *O (N^2)*

Complejidad: *O (N^2)*(Tiempo cuadratico).

Fragmento 4:
Python

```
For i = 0 to length(A):
  for j = 0 to length(B):
    print A[i] + “,” + B[j]
```

Analisis: los dos bucles están anidados, pero dependen de dos entradas con tamaños potencialmente distintos *(length (A) y length (B).* Definamos *N= length(A) y M= length(B).* el bucle interno corre *M* veces por cada una de las *N* iteraciones del bucle externo.

Función: *O (N x M)* (

Complejidad: *O (N x M) (*Tiempo cuadrático de dos variables; si asumimos que *A y B* tienen igual longitud *N*, se simplifica a *O (N^2)*)

Fragmento 5:
Python

```
I = 1
while (i < N):
  print i
  i = i * 2
```

Análisis: en cada paso del ciclo, la variable de control **i** se multiplica por 2 **(1,2,4,8,16…).** Esto significa que el numero se duplica (o mirándolo a la inversa, la distancia hacia *N* se reduce a la mitad de forma geométrica), lo cual corresponde estrictamente a una progresión logarítmica.

Función: $O(log_2 N)$

Complejidad: *O (log N*) (Tiempo Logarítmico).

Fragmento 6:
Python

```
For i = 0 to N:
  j 0 1
  while j < N:
     print j
     j = j * 2
```

Analisis: tenemos un bucle while logarítmico (idéntico al punto anterior, que toma *O (log N*) anidado dentro de un bucle **for** externo que corre linealmente *N* veces.

Funcion: *N x O (log N*)

Complejidad: *O (log N*) (Tiempo Logaritmico)

Fragmento Extra (numerado abajo como otro 6)
Python

```
N = 1000
if N % 2 == 0:
   print “par”
else:
  print “impar”
```

Analisis: no existen estructuras de repetición (bucles) ni llamadas que dependan de una entrada variable; la asignación y la condicional se evalúan una cantidad fija de veces de forma directa.

Función: *O (1*)

Complejidad: *O (1*)(Tiempo Constante).